

HOUSE SERVICE

Plataforma de Serviços Domésticos

DOCUMENTO DE REQUISITOS E ESPECIFICAÇÃO TÉCNICA

Versão	1.0
Data	Junho de 2025
Tecnologias	HTML5 CSS3 JavaScript
Tipo	Landing Page — Single Page Application
Status	Produção

1. VISÃO GERAL DO PROJETO

1.1 Descrição

O House Service é uma plataforma digital do tipo Single Page Application (SPA) desenvolvida com tecnologias web nativas: HTML5, CSS3 e JavaScript puro. Seu objetivo é conectar clientes a profissionais domésticos verificados — como diaristas, faxineiras, cuidadores, jardineiros, cozinheiros e técnicos de manutenção — de forma segura, rápida e transparente.

1.2 Objetivos da Plataforma

- Oferecer uma vitrine digital profissional para o serviço House Service
- Apresentar os tipos de serviços disponíveis de forma clara e organizada
- Transmitir confiança ao usuário por meio de sistemas de reputação e verificação
- Converter visitantes em clientes ou prestadores cadastrados
- Funcionar corretamente em qualquer dispositivo (desktop, tablet e celular)

1.3 Público-Alvo

Perfil	Descrição
Clientes	Pessoas que buscam profissionais domésticos de confiança para suas residências
Profissionais	Prestadores de serviço que desejam divulgar seu trabalho e construir reputação
Novos usuários	Pessoas conhecendo a plataforma pela primeira vez via busca ou indicação

2. ESTRUTURA DE ARQUIVOS E ORGANIZAÇÃO

2.1 Organização do Projeto

O projeto segue uma estrutura simples, direta e padronizada, separando responsabilidades entre os três arquivos principais:

house-service/

houseservice.html — Estrutura e conteúdo da página houseservice.css — Estilos, layout e animações visuais houseService.js — Comportamentos, interações e animações dinâmicas

2.2 Justificativa da Separação

Arquivo	Responsabilidade
houseservice.html	Define toda a estrutura semântica da página: seções, textos, botões, links e hierarquia de conteúdo
houseservice.css	Controla 100% da apresentação visual: cores, tipografia, espaçamentos, grid, hover e animações CSS
houseService.js	Gerencia todos os comportamentos dinâmicos: scroll, animações por observação, contadores e rolagem suave

Esta separação é uma boa prática fundamental no desenvolvimento web, conhecida como Separação de Responsabilidades (Separation of Concerns). Cada arquivo tem uma função clara e exclusiva, o que facilita manutenção, leitura e escalabilidade do código.

3. RECURSOS E FUNCIONALIDADES

3.1 Seções da Página

Seção	Descrição
Navegação (Nav)	Menu fixo no topo com links internos para todas as seções e botão de agendamento
Hero	Apresentação principal com título, descrição, botões de ação e indicadores de confiança
Banner de Estatísticas	Faixa com números-chave: profissionais, serviços realizados, avaliação média e cidades
Serviços	Grid com 6 categorias de serviços disponíveis na plataforma
Como Funciona	Passo a passo de 4 etapas explicando o processo de contratação
Reputação	Seção escura explicando o sistema de crescimento por mérito dos profissionais
Planos	Comparativo dos 3 planos disponíveis: Gratuito, Mensal e Profissional
Avaliações	Depoimentos reais de clientes e profissionais que usam a plataforma
CTA Final	Chamada para ação com dois caminhos: cliente ou profissional
Rodapé	Links organizados por categoria, logo e informações da empresa

3.2 Funcionalidades Interativas

Funcionalidade	Descrição Técnica
Navbar com efeito scroll	Adiciona sombra ao menu quando o usuário rola mais de 20px usando <code>classList.toggle</code>
Scroll Reveal	Elementos aparecem com animação suave ao entrar na área visível via <code>IntersectionObserver</code>
Passos em sequência	Os 4 passos do Como Funciona surgem um após o outro com intervalo de 150ms via <code>setTimeout</code>
Barras de reputação	Barras crescem até o valor definido em <code>data-width</code> quando a seção é visualizada
Contadores animados	Números contam do zero ao valor final com <code>setInterval</code> ao entrar na viewport
Rolagem suave	Cliques nos links do menu rolam suavemente até a seção correspondente
Hover nos cards	Cards de serviço elevam e exibem linha verde ao receber hover via CSS

4. DECISÕES DE LAYOUT E INTERFACE

4.1 Sistema de Cores

A paleta de cores foi definida via variáveis CSS no seletor `:root`, garantindo consistência em toda a página e facilitando futuras alterações:

Variável	Valor e Uso
<code>--teal</code>	#00BFA6 — Cor primária: botões, destaques, ícones e elementos ativos
<code>--teal-dark</code>	#009688 — Hover e versão mais escura da cor primária
<code>--teal-light</code>	#E0F7F4 — Fundo suave para badges e áreas de destaque
<code>--navy</code>	#0D2B3E — Cor principal de textos, títulos e seções escuras
<code>--navy-mid</code>	#1A3F57 — Variação intermediária para fundos e cards escuros
<code>--text-muted</code>	#6B8A99 — Textos secundários, descrições e legendas
<code>--border</code>	#D6ECEA — Bordas suaves dos cards e separadores

4.2 Tipografia

Fonte	Aplicação
Syne (700/800)	Títulos, headings, números de destaque e logotipo — comunicação de força e modernidade
DM Sans (300/400/500)	Corpo de texto, descrições e botões — leitura fluida e leveza visual

Ambas as fontes são importadas do Google Fonts no `<head>` do HTML, garantindo carregamento externo sem dependência de instalação local.

4.3 Sistema de Grid e Layout

Contexto	Decisão de Layout
Hero principal	CSS Grid com 2 colunas iguais (1fr 1fr): texto à esquerda, visual à direita
Grid de serviços	auto-fit com minmax(220px, 1fr): adapta o número de colunas ao espaço disponível
Como Funciona	Grid de 2 colunas: passos à esquerda, cards de perfil à direita
Planos	Grid fixo de 3 colunas para comparação lado a lado
Avaliações	Grid fixo de 3 colunas para depoimentos em paralelo

Rodape	Grid de 4 colunas: marca (2fr) + 3 colunas de links (1fr cada)
Banner de stats	Grid de 4 colunas iguais com separadores verticais

4.4 Responsividade

A página implementa breakpoint único em 900px via @media query, adaptando todos os grids para coluna única em dispositivos menores:

- Hero: passa de 2 colunas para 1 coluna; visual e oculto
- Como Funciona e Reputação: grids colapsam para 1 coluna
- Planos e Avaliações: cards empilham verticalmente
- Rodape: reorganiza para 2 colunas
- Banner de stats: passa para 2 colunas
- Menu de navegação: links ocultos para economia de espaço

5. TECNOLOGIAS APLICADAS

5.1 Stack Tecnológica

Tecnologia	Versão / Padrão	Função no Projeto
HTML5	W3C — atual	Estrutura semântica de toda a página
CSS3	W3C — atual	Estilização, layout, animações e responsividade
JavaScript	ES6+ (Vanilla)	Comportamentos dinâmicos e interações do usuário
Google Fonts	API externa	Importação das fontes Syne e DM Sans
IntersectionObserver API	Web API nativa	Deteção de elementos na viewport
CSS Grid	CSS3 Module	Sistema de layout principal da página
CSS Flexbox	CSS3 Module	Alinhamento de componentes internos
CSS Custom Properties	CSS3 Module	Sistema de variáveis globais de design

5.2 Implementação do HTML5

O HTML utiliza elementos semânticos corretos para cada tipo de conteúdo:

Elemento	Uso no Projeto
<nav>	Menu de navegação principal com id para acesso via JS
<section>	Cada bloco de conteúdo da página (hero, serviços, planos, etc.)
<h1>, <h2>	Hierarquia de títulos para SEO e acessibilidade
/	Listas de features dos planos e links do rodapé
<footer>	Rodapé com informações institucionais e links
<button>	Ações primárias como agendamento e assinatura de planos
<a>	Links de navegação interna (#âncora) e externa
data-width	Atributo customizado para armazenar valor das barras de reputação

5.3 Implementação do CSS3

Recurso CSS	Implementação no Projeto
Custom Properties (:root)	14 variáveis de cor, sombra e radius usadas em todo o arquivo
CSS Grid	Layout principal de 6 seções diferentes com configurações próprias
Flexbox	Alinhamento interno de nav, cards, botões e itens de lista
@keyframes	3 animações: fadeInUp, fadeInRight e float (flutuação do hero)
Pseudo-elementos ::before::after	Efeitos decorativos em hero, CTA e linha nos service-cards
Pseudo-classe :hover	Feedback visual em cards, botões e links
backdrop-filter: blur	Efeito glassmorphism no menu de navegação
clamp()	Tamanho de fonte responsivo sem media queries nos títulos principais
@media query	Breakpoint em 900px para adaptação mobile
transition	Suavização de todas as interações de hover e animações de estado

5.4 Implementação do JavaScript

Recurso JS	Implementação no Projeto
IntersectionObserver	3 instâncias: reveal geral, passos em sequência e barras de reputação
addEventListener scroll	Monitoramento de rolagem para efeito do navbar
classList.toggle / add	Ativação e desativação de classes CSS via JS
querySelectorAll / forEach	Seleção e iteração sobre múltiplos elementos do DOM
setTimeout	Atraso escalonado de 150ms para surgimento sequencial dos passos
setInterval / clearInterval	Controle do contador animado: incremento a cada 25ms com parada precisa
getAttribute	Leitura do atributo data-width para animação das barras
scrollIntoView	Rolagem suave até âncoras com behavior: smooth
toLocaleString('pt-BR')	Formatação dos números no padrão brasileiro (ponto como separador de milhar)

6. BOAS PRÁTICAS APLICADAS

6.1 Código HTML

- Separação total de estrutura (HTML), apresentação (CSS) e comportamento (JS)
- Uso de elementos semânticos corretos para cada tipo de conteúdo
- IDs e classes com nomes descritivos e padrão kebab-case (ex: hero-card-main, nav-cta)
- Atributos data-* para comunicação de dados entre HTML e JS sem poluir o DOM
- Fontes externas carregadas no <head> para evitar flash de texto sem estilo (FOUT)
- Links internos com # para navegação dentro da mesma página sem recarregamento

6.2 Código CSS

- Variáveis CSS centralizadas no :root para consistência e fácil manutenção
- Reset global com * { box-sizing: border-box; margin: 0; padding: 0 }
- Uso de clamp() para tipografia fluida sem necessidade de múltiplas media queries
- Animações com transform e opacity — propriedades que não causam reflow no navegador
- Classe .reveal reutilizável em qualquer elemento que precise de animação de entrada
- Media query única e clara em 900px para toda a responsividade do projeto
- Variáveis semânticas: --shadow, --radius, --border, facilitando consistência visual

6.3 Código JavaScript

- Uso de IntersectionObserver em vez de cálculo manual de posição no evento scroll
- Verificação de existência de elemento antes de observar (if (element) observer.observe(...))
- statsObserver com unobserve() após primeira execução, evitando animação repetida
- Função animateCounter reutilizável com parâmetros target e suffix
- Separação de lógicas: cada Observer tem responsabilidade única e clara
- Uso de clearInterval para garantir parada precisa dos contadores
- preventDefault() no clique dos links para controlar manualmente a rolagem

7. FUNCIONAMENTO E FLUXO DA PAGINA

7.1 Fluxo de Carregamento

Etapa	O que acontece
1. HTML carregado	Estrutura da pagina e renderizada; CSS e JS são referenciados no <head> e ao final do <body>
2. CSS aplicado	Todos os estilos são aplicados; elementos .reveal ficam opacity: 0 e deslocados
3. JS executado	Observers são registrados; evento de scroll e listener do navbar são ativados
4. Usuario rola	IntersectionObserver detecta elementos visíveis e adiciona classe .visible
5. Animacoes disparam	CSS transiciona opacity e transform suavemente em 0.6s
6. Secao de stats visivel	Contadores começam a animar do zero até o valor final
7. Secao de reputacao visivel	Barras crescem até a porcentagem definida no data-width

7.2 Conexão entre os Arquivos

Os três arquivos se comunicam por meio de classes e IDs do HTML:

HTML define → CSS estiliza → JS ativa

Exemplo: a classe .reveal é declarada no HTML, estilizada no CSS (invisível por padrão) e ativada pelo JS (adiciona .visible quando o elemento aparece na tela). Esse é o padrão central que une os três arquivos.

7.3 Verificação de Funcionamento

Funcionalidade	Resultado Esperado
Abrir a pagina	Hero aparece com animação fadeInUp no conteúdo e fadeInRight no visual
Rolar a pagina	Elementos .reveal surgem suavemente conforme aparecem na tela
Passar mouse nos cards	Card sobe 6px, sombra aparece e linha teal surge na base
Clicar em link do menu	Pagina rola suavemente até a seção correspondente
Chegar ao banner de stats	Números contam do zero até o valor final em ~1.5 segundos
Chegar na seção Reputacao	Barras crescem da esquerda para a direita até as porcentagens definidas
Rolar mais de 20px	Menu ganha sombra sutil na parte inferior

Acessar em celular

Layout reorganiza para coluna única; menu de links some

8. DOCUMENTAÇÃO DE COMPONENTES

8.1 Componentes Visuais

Componente	Descrição
<code>.hero</code>	Seção principal com grid de 2 colunas, gradiente de fundo e pseudo-elementos decorativos
<code>.service-card</code>	Card com hover elevado, linha teal animada via <code>::before</code> e ícone com transição de cor
<code>.plan-card.featured</code>	Card de plano destacado com fundo teal, badge 'Mais Popular' e botão invertido
<code>.profile-card-big</code>	Card de profissional com avatar, nome, função, estrelas e badge de verificação
<code>.rep-bar</code>	Barra de progresso animada: largura definida em <code>data-width</code> no HTML, ativada pelo JS
<code>.mini-card</code>	Card pequeno do hero com ícone, título e descrição resumida do diferencial
<code>.hero-overlay-card</code>	Card flutuante sobre o visual do hero com glassmorphism e dados do profissional
<code>.stats-banner</code>	Faixa teal com grid de 4 colunas separadas por linhas verticais semitransparentes

8.2 Classes Utilitárias

Classe	Função
<code>.reveal</code>	Elemento começa invisível e animado; JS adiciona <code>.visible</code> quando entra na viewport
<code>.visible</code>	Estado ativo: <code>opacity: 1</code> e <code>transform: translateY(0)</code> — aplicada pelo JS
<code>.scrolled</code>	Adicionada ao navbar pelo JS após 20px de scroll; ativa <code>box-shadow</code> no CSS
<code>.section-label</code>	Etiqueta superior das seções em maiúsculo, cor teal e letra espacada
<code>.section-title</code>	Título principal de cada seção com fonte Syne, peso 800 e cor navy
<code>.section-sub</code>	Subtítulo de cada seção com texto muted, <code>max-width</code> de 500px e peso 300
<code>.btn-primary</code>	Botão principal com fundo teal, sombra e hover com elevação
<code>.btn-outline</code>	Botão secundário transparente com borda e hover teal

9. RESUMO EXECUTIVO

O projeto House Service é uma landing page profissional desenvolvida com HTML5, CSS3 e JavaScript puro, sem frameworks ou dependências externas além do Google Fonts. A implementação segue boas práticas consolidadas de desenvolvimento web, com separação clara de responsabilidades entre os três arquivos.

O código é limpo, legível e bem organizado. As decisões de layout priorizam a experiência do usuário — com animações suaves, responsividade completa e interações intuitivas. O sistema de variáveis CSS garante consistência visual e facilita manutenção futura.

A plataforma cumpre seu objetivo principal: apresentar o serviço House Service de forma convincente, transmitir confiança ao usuário e convertê-lo em cliente ou profissional cadastrado.

Critério	Avaliação	Observação
Corretude do código	Aprovado	HTML, CSS e JS válidos e funcionais
Organização de arquivos	Aprovado	Separação clara em 3 arquivos com responsabilidades distintas
Boas práticas	Aprovado	Reset, variáveis, semântica e reutilizabilidade implementados
Código limpo	Aprovado	Nomenclatura descritiva, sem código morto ou redundante
Funcionamento da página	Aprovado	Todas as interações e animações funcionam corretamente
Responsividade	Aprovado	Funciona em desktop, tablet e celular
Documentação	Aprovado	Este documento cobre todos os aspectos do projeto
Tecnologias padronizadas	Aprovado	HTML5, CSS3, JS ES6+ com APIs nativas do navegador